OG-NSD: Neuro-Symbolic Ontology Drafting from Natural-Language Requirements

Efstratios Skaperdas
Kristi Cami
Nick Bassiliades
Department of Informatics, Aristotle University of Thessaloniki, Greece
Greece

Abstract

Natural-language requirements are easy for analysts and domain stakeholders to write, but difficult to validate systematically and to transform into machine-interpretable semantic artifacts. We present *OG-NSD*, a neuro-symbolic pipeline for ontology drafting from natural-language requirements. The pipeline combines LLM-based drafting with ontology-aware prompting, SHACL validation, OWL/DL reasoning, competency-question evaluation, and an iterative repair loop that converts symbolic violations into corrective prompts. We evaluate the approach through a controlled pilot study spanning neural baselines, a validation-oriented baseline, ontology-aware drafting, iterative repair policies, cross-domain transfer, and CQ-oriented iterative analysis. The results show a deliberately mixed but informative picture. Low-temperature drafting provides a strong raw baseline ($F1 = 0.4769$, 11/21 competency questions), ontology-aware prompting improves vocabulary alignment but not necessarily precision, and iterative repair helps only when conservative stopping policies are used. Among the tested repair strategies, the `max_only` configuration yields the best balance within the iterative family ($F1 = 0.3427$, 11/21 competency questions), whereas aggressive CQ-driven repair leads to graph explosion and unstable reasoning. These findings support the main claim of the paper: symbolic feedback is most useful when treated as an active control mechanism for ontology drafting rather than as a purely post-hoc diagnostic layer.

Keywords

LLMs, ontology engineering, requirements formalization, neuro-symbolic AI, SHACL, OWL

1 Introduction

Requirements are commonly expressed in natural language because this format is accessible to analysts, developers, and domain stakeholders. However, natural-language requirements are often ambiguous, underspecified, and difficult to validate systematically. This limits downstream automation, including traceability, consistency checking, reuse, and formal analysis. In ontology engineering, the problem is particularly significant: requirement documents may implicitly define classes, relations, hierarchies, and domain constraints, yet converting them into formal semantic artifacts remains a labor-intensive and expert-driven task [1, 2].

Large language models (LLMs) have recently shown strong capabilities for extracting structured information from free text and producing draft knowledge representations. This raises a natural question: can LLMs accelerate ontology drafting directly from requirement descriptions? The answer is promising, but not straightforward. In practice, LLM outputs may omit constraints, invent classes or properties, drift away from domain vocabulary, or produce ontologies that are syntactically plausible but semantically weak. For ontology engineering, such defects are critical because quality is not measured only by fluent output, but by structural validity, logical consistency, and the ability to support intended queries [8, 9, 11].

In this paper, we present *OG-NSD* (Ontology-Guided Neuro-Symbolic Drafting), a pipeline for transforming natural-language requirements into draft OWL ontologies through a closed neuro-symbolic loop. The key idea is simple: an LLM produces a candidate ontology, symbolic validators detect structural and logical problems, and these signals are transformed into targeted corrective prompts that guide the next repair step. Rather than treating validation as a final filtering stage, OG-NSD incorporates validation into generation itself.

The contribution of this work is fourfold:

- We define a modular neuro-symbolic pipeline for LLM-driven ontology drafting from requirements.
- We operationalize SHACL violations, reasoning outcomes, and competency-question failures as structured feedback for iterative repair.
- We present a controlled experimental suite (E1–E6) that separates the effects of raw drafting, symbolic control, ontology-aware prompting, iterative repair, cross-domain transfer, and CQ-oriented iterative analysis.
- We provide a realistic pilot evaluation over ATM and healthcare requirements, showing both the strengths and the limits of the approach.

The paper intentionally makes a measured claim rather than a maximalist one. We do not claim that the full neuro-symbolic loop uniformly dominates every baseline under every metric. Instead, the experiments show a more nuanced picture: low-temperature drafting is already strong, ontology-aware prompting improves alignment but can still introduce structural noise, and repair helps only when its stopping policy is well controlled. That empirical profile is precisely what makes the problem scientifically interesting.

2 Related Work

The proposed work lies at the intersection of ontology engineering, natural-language requirements formalization, symbolic validation, and LLM-assisted structured generation.

Conference'17, Washington, DC, USA
2026. ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

A first line of work concerns the role of ontologies as formal, reusable representations of domain knowledge. Foundational work by Gruber framed ontologies as explicit specifications of conceptualizations and emphasized portability and reuse [1]. Uschold and Grüninger further systematized ontology development by discussing principles, methods, and application contexts [2]. In ontology engineering practice, competency questions (CQs) became a standard way to delimit ontology scope and evaluate adequacy with respect to intended tasks [3, 4].

A second line of work concerns the formal languages and validation mechanisms used to specify and check ontologies. OWL 2 provides the standard semantic framework for ontology representation on the Semantic Web [6, 7]. SHACL provides a graph-based validation language for checking RDF data against structural and typing constraints [5]. These technologies offer strong guarantees at the symbolic level, but they are usually applied after ontology construction rather than integrated into the generation loop.

A third line of work explores LLMs for ontology engineering and ontology learning. Recent studies report that LLMs can assist with relation extraction, glossary induction, ontology extension, schema suggestion, and direct ontology generation from textual descriptions [8–11]. At the same time, these works also highlight recurring weaknesses: hallucinated schema elements, inconsistent use of vocabulary, weak traceability to source text, and difficulty maintaining structural discipline across domains.

Finally, recent neuro-symbolic perspectives argue that symbolic constraints should not be treated merely as post-hoc diagnostics, but as mechanisms that actively shape neural generation and reasoning [12]. OG-NSD aligns with this direction while focusing specifically on ontology drafting from requirement sets and on the conversion of symbolic diagnostics into repair-oriented prompts.

3 OG-NSD Method

3.1 Problem Setting

Given a set of natural-language requirements $R = \{r_1, \ldots, r_n\}$ and optional domain assets A (e.g., base ontology, SHACL shapes, competency questions, glossary terms, and exemplar mappings), the goal is to generate a draft ontology O in OWL/Turtle that captures the key classes, properties, hierarchies, and constraints needed to support downstream validation and querying.

We assume a practical setting in which exact gold formalizations may be incomplete or domain-dependent. Therefore, across the full experimental suite, ontology quality is not reduced to exact overlap with a reference ontology, but is assessed more broadly through structural validation, reasoning-based checks, and competency-question performance where applicable.

3.2 Inputs and Assets

The implementation relies on a clearly defined set of inputs and supporting assets. Requirement files are stored in JSONL or JSON form, with each requirement processed as a standalone natural-language statement. The pipeline also accepts base ontologies, SHACL shapes, competency-question files, configuration files, and optional gold ontologies for evaluation. This design supports reproducibility and allows experiments to be replayed without modifications to the core code.

The main asset categories are:

- **Requirements**: JSONL/JSON files containing requirement statements, such as `atm_requirements.jsonl` and `health_requirements.jsonl`.
- **Domain ontology**: optional base ontology used for grounding and graph merging.
- **SHACL shapes**: domain-specific structural constraints encoded in Turtle.
- **Competency Questions**: SPARQL ASK queries used to assess conceptual adequacy.
- **Ontology context and exemplars**: optional vocabulary, prefixes, and small Turtle exemplars injected into prompts during ontology-aware drafting.
- **Configuration files**: experiment-specific parameter snapshots controlling LLM mode, reasoning, repair iterations, and paths.
- **Outputs and reports**: TTL ontologies, JSON metrics, and intermediate artifacts stored under `build/` or `runs/`.

Figure 1 summarizes the overall OG-NSD workflow and the role of the main assets used throughout the pipeline. The figure makes explicit that ontology drafting is not treated as a one-shot generation task: requirements, domain vocabulary, SHACL shapes, and competency questions jointly support a closed validation-and-repair process.

As Figure 1 suggests, the method is organized around a separation of concerns: drafting expands coverage, validation enforces discipline, and repair controls the interaction between the two. This decomposition is important because it lets us study where errors originate and which part of the pipeline is actually responsible for improvements.

3.3 Preprocessing

Preprocessing in OG-NSD is intentionally lightweight. The system performs syntactic loading, whitespace normalization, filtering of empty or malformed entries, and optional chunking of requirement sets. The design avoids aggressive linguistic transformations such as stemming or lemmatization, because the original requirement formulations are treated as the semantic ground truth.

Requirement loading is implemented so that the rest of the pipeline remains format-agnostic. The requirement loader supports both JSONL, where each line is an independent requirement, and JSON lists. In both cases, the output is normalized into a uniform list of strings. Large requirement sets can be chunked into batches in order to respect LLM context limits and to keep execution cost under control. This chunking step functions as an operational split mechanism rather than as a train/validation/test split. It allows the same requirement subset to be used across multiple runs, thus improving reproducibility and comparability.

The same stage also supports the injection of ontology vocabulary and exemplars into prompts. These exemplars are small Turtle fragments that illustrate how requirement phrases should be translated into ontology constructs. They function as a style guide for namespace usage, subclass modeling, and property declarations.

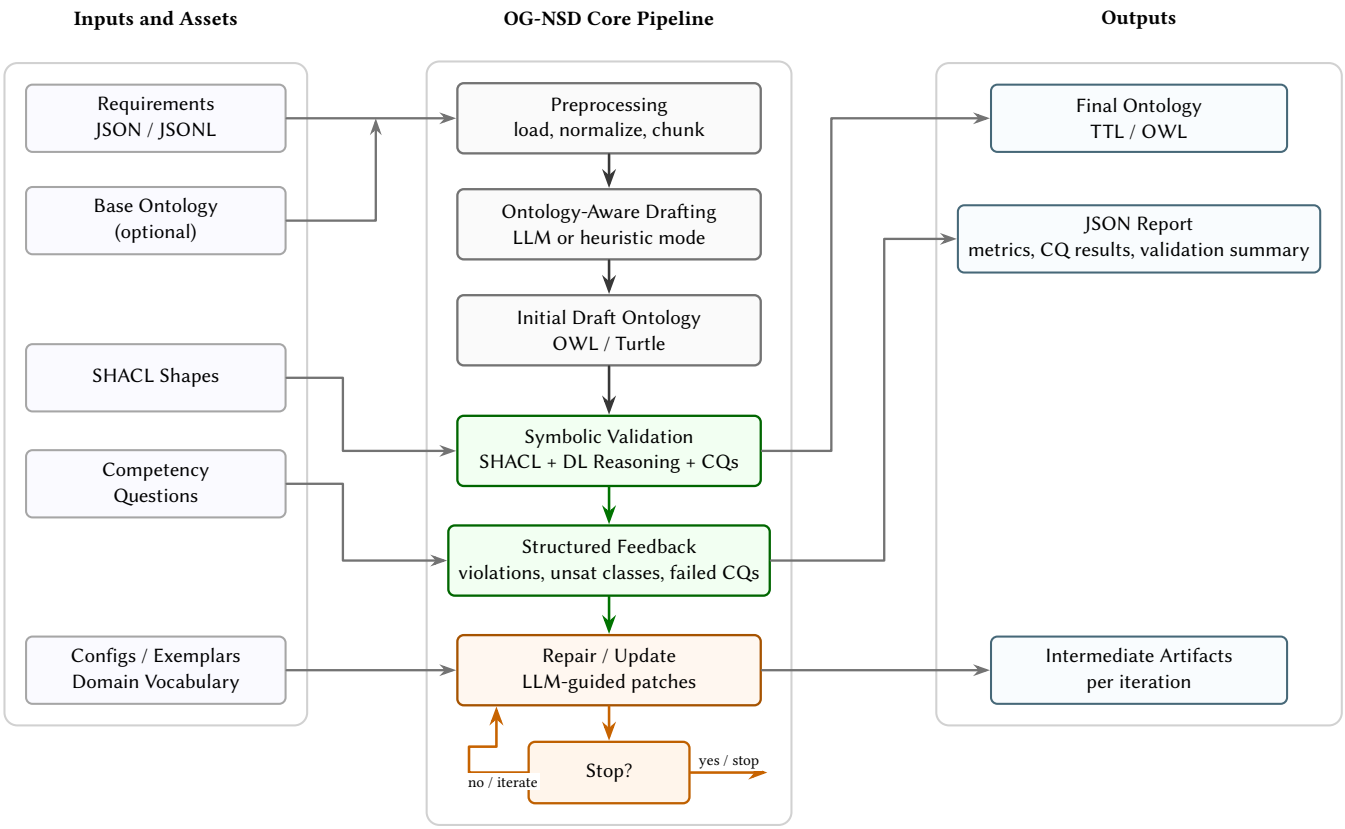

Figure 1: Overview of the OG-NSD pipeline. Natural-language requirements and domain assets are transformed into a draft ontology through ontology-aware drafting. The candidate ontology is then symbolically validated using SHACL, DL reasoning, and competency questions, and iteratively refined through structured feedback until the stopping criteria are satisfied.

3.4 Configuration, Model Settings, and Reproducibility

The behavior of the pipeline is controlled through a unified configuration layer. A configuration object specifies paths to requirements, gold ontologies, SHACL shapes, competency questions, and optional base ontologies, as well as operational parameters such as LLM mode, maximum number of repair iterations, maximum number of requirements to process, whether reasoning is enabled, and whether the run should stop after drafting.

This design supports two desirable properties: reproducibility, because each experiment corresponds to a fixed configuration snapshot, and comparability, because differences between runs can be attributed to explicit parameter changes rather than to hidden code modifications.

The implementation consists of modules for requirement loading and chunking, LLM drafting, ontology parsing and serialization, SHACL validation, DL reasoning, CQ execution, and JSON reporting. The drafting stage supports both a live LLM backend and a heuristic fallback mode. In the reported experiments, the neural baseline explicitly studies two temperatures ($T = 0.1$ and $T = 0.7$), allowing us to quantify the effect of generation stochasticity. Ontology-aware runs use domain vocabulary and exemplar-guided prompting. Full repair experiments are executed under fixed stopping policies, with maximum iteration budgets defined in the corresponding configuration files. All runs produce serialized ontologies together with JSON summaries, which makes the evolution from initial draft to final output auditable and directly reproducible.

The public implementation and experimental artifacts are available in the project repository at https://github.com/KristiCami/Ontology-Guided. The released experiment scripts support both OpenAI-backed execution and a deterministic heuristic fallback. In the public code, the effective backend, temperature, ontology-context usage, reasoning flag, and iteration budget are selected through configuration files, while the generated run artifacts record the conditions of each execution. We therefore interpret the reported runs through their configuration snapshots and saved outputs rather than assuming a single fixed backend across all experiments. For the reported OpenAI-backed runs, the provider, model name, decoding settings, and the use of the same or different models across drafting and repair are specified in the corresponding configuration snapshots and documented in the released artifacts.

3.5 LLM Drafting

The drafting stage is the neural core of OG-NSD. Its input is a set of normalized requirements, and its output is an initial draft ontology in Turtle. The system supports two execution modes: an OpenAI-backed mode and a heuristic fallback mode. In the public implementation, the effective LLM mode and temperature are selected through configuration, while the heuristic mode remains useful for offline runs and deterministic reproduction.

Ontology-aware prompting is a key feature of the system. When enabled, the pipeline extracts vocabulary from a gold or base ontology—classes, properties, domains, ranges, and prefixes—and injects this information into the prompt as structured context. This does not amount to copying ontology axioms; rather, it acts as a lexical and structural contract that reduces drift and promotes consistent namespace usage. The resulting draft is serialized and stored as a temporary artifact so that its contribution can be studied independently before symbolic correction is applied.

3.6 Validation and Repair Loop

After drafting, the candidate ontology is validated at multiple levels.

- **SHACL validation.** SHACL captures structural expectations such as missing properties, invalid typing, incomplete declarations, and domain-specific modeling rules. The resulting violations are fine-grained and interpretable, making them suitable as repair signals.
- **DL reasoning.** Reasoning checks whether the ontology remains logically coherent under OWL semantics. It can reveal inconsistencies and unsatisfiable classes that are not visible through purely local shape checks.
- **Competency-question testing.** Competency questions are implemented as SPARQL ASK queries. They provide an independent signal of domain adequacy by checking whether the ontology supports intended domain questions.

The outputs of these validators are then transformed into repair-oriented feedback. Instead of exposing raw tool logs to the model, OG-NSD produces compact structured prompts that specify the problem, its local context, and the expected correction. Let $O^{(t)}$ denote the ontology at iteration t. The repair step is written as:

$$O^{(t+1)} = \text{RepairLLM}(O^{(t)}, V^{(t)}_{\text{shacl}}, V^{(t)}_{\text{reason}}, C^{(t)}),$$

where $V^{(t)}_{\text{shacl}}$ and $V^{(t)}_{\text{reason}}$ denote validation signals and $C^{(t)}$ denotes optional competency-question feedback.

Each repair iteration includes four substeps: validation, violation aggregation, repair-prompt synthesis, and patch integration into the current graph. The process is therefore closer to counterexample-guided repair than to naive prompt repetition. The entire iteration history is stored in JSON form, making it possible to inspect what changed and why.

3.7 Termination, Serialization, and Reporting

The repair loop terminates when no hard SHACL violations remain, when CQ performance stabilizes, when no new patches are produced, or when the maximum number of iterations is reached. These criteria balance quality improvement against runtime and LLM cost.

The final ontology is serialized to Turtle via an ontology abstraction layer built on top of RDFLib. The serialized graph may combine the initial draft, an optional base ontology, and the patches introduced during repair. Intermediate graphs and JSON reports are preserved so that the evolution from draft to final ontology remains fully traceable. As a result, the output of a run is not a single ontology file, but a pair consisting of the final TTL graph and an analytical report containing SHACL summaries, reasoner results, CQ traces, and repair metadata.

3.8 Robustness and Auditability

A further implementation detail that is important for interpreting the experiments is that OG-NSD is designed to fail transparently rather than silently. Requirement files are normalized before drafting, Turtle outputs are parsed before downstream validation, and malformed or invalid restrictions are explicitly recorded in the run artifacts rather than being hidden inside later stages. Likewise, intermediate outputs are preserved per iteration so that one can distinguish defects introduced during initial drafting from defects created or amplified during repair.

This matters experimentally for two reasons. First, it improves auditability: when a repair policy behaves poorly, the analyst can inspect the exact draft, validation output, patch set, and post-reasoning graph associated with that run. Second, it improves comparability: because execution is configuration-driven and artifact-producing, differences between methods can be tied to concrete choices such as stopping policy, threshold setting, or prompting mode. In practice, this logging discipline is especially useful in failure cases such as graph explosion, repeated invalid restrictions, or reasoner instability, where a pure black-box evaluation would reveal that something went wrong but not where it went wrong.

3.9 Evaluation Methodology

The evaluation of generated ontologies in OG-NSD is explicitly multi-layered. It combines four complementary perspectives.

First, exact matching compares predicted axioms against the reference ontology at strict IRI level after duplicate removal and namespace normalization. Second, we report a relaxed reasoning-aware overlap metric. This metric uses the reasoned graph when reasoning is enabled and otherwise the asserted graph, and compares a normalized schema-oriented triple space. It should therefore be interpreted as an approximate indicator of semantic proximity rather than as full semantic equivalence. Third, structural and logical quality are measured through SHACL and DL reasoning. Fourth, competency questions assess whether the ontology captures the domain knowledge needed to support intended queries.

This layered design is necessary because no single metric captures ontology quality in full. An ontology may be syntactically close to the gold reference while remaining logically weak, or it may be logically valid but still fail to support the intended domain questions.

References

[1] T. R. Gruber. A translation approach to portable ontology specifications. *Knowledge Acquisition*, 5(2):199–220, 1993.
[2] M. Uschold and M. Grüninger. Ontologies: Principles, methods and applications. *The Knowledge Engineering Review*, 11(2):93–136, 1996.

[3] M. Grüninger and M. S. Fox. The role of competency questions in enterprise engineering. In *Benchmarking — Theory and Practice*, pages 22–31. Springer, 1995.
[4] G. K. Q. Monfardini, J. P. A. Almeida, and G. Guizzardi. Use of competency questions in ontology engineering: A survey. *Applied Ontology*, 18(1):1–38, 2023.
[5] H. Knublauch and D. Kontokostas, editors. *Shapes Constraint Language (SHACL)*. W3C Recommendation, 2017.
[6] W3C OWL Working Group. *OWL 2 Web Ontology Language: Document Overview (Second Edition)*. W3C Recommendation, 2012.
[7] B. Motik, P. F. Patel-Schneider, and B. Parsia, editors. *OWL 2 Web Ontology Language: Structural Specification and Functional-Style Syntax (Second Edition)*. W3C Recommendation, 2012.
[8] J. García Fernández. *Ontology Engineering with Large Language Models*. Master's thesis, Delft University of Technology / TNO, 2024.
[9] M. J. Saeedizade, A. S. Lippolis, M. Poveda-Villalón, and O. Corcho. Navigating ontology development with large language models. In *Proceedings of ESWC 2024 Workshops*, 2024.
[10] A. S. Lippolis, M. J. Saeedizade, M. Poveda-Villalón, and O. Corcho. Ontology generation with metacognitive prompting in large language models. In *Proceedings of ESWC 2024 Workshops*, 2024.
[11] D. Garijo, M. Poveda-Villalón, E. Amador-Domínguez, Z. Wang, R. García-Castro, and O. Corcho. LLMs for ontology engineering: A landscape of tasks and benchmarking challenges. In *Proceedings of the 1st International Workshop on LLMs for Ontology Engineering*, 2025.
[12] L. N. DeLong, R. Fernández Mir, and J. D. Fleuriot. Neurosymbolic AI for reasoning over knowledge graphs: A survey. *arXiv preprint arXiv:2302.07200*, 2024.